

HMAC SystemC TLM High-Level Design Specification

Document Version: 1.0 **Date:** 2025-10-23 **IP Module:** HMAC (Hash-based Message Authentication Code) **Abstraction Level:** SystemC TLM2.0

Table of Contents

1. Introduction
 2. Design Overview
 3. Features
 4. Configuration Parameters
 5. Port Interfaces
 6. Memory-Mapped Registers
 7. Register Callbacks
 8. Modeling Assumptions
 9. Summary and Conclusions
-

1. Introduction

1.1 Purpose

This document provides a comprehensive high-level design specification for the HMAC (Hash-based Message Authentication Code) peripheral implemented as a SystemC Transaction Level Model (TLM2.0). The HMAC peripheral provides hardware-accelerated secure hashing and message authentication capabilities for cryptographic applications in virtual platform environments.

1.2 Scope

This specification covers:

- Functional features and operational modes of the HMAC peripheral
- Build-time configuration parameters affecting model structure and behavior
- External port interfaces for virtual platform integration
- Complete memory-mapped register definitions
- Register callback requirements for functional side-effects
- Modeling assumptions defining abstraction boundaries

This document serves as the primary reference for SystemC TLM model implementation, verification, and integration into virtual platforms.

1.3 Architectural Context

Critical Architectural Decision: The SystemC TLM model orchestrates data flow, register access, and status management while delegating all cryptographic computations to the opensource Botan library. The model focuses on:

- Software-visible register interface and control flow
- Transaction-level timing characteristics
- State machine management and access control
- Error detection and interrupt generation
- Message FIFO management and back-pressure handling

The Botan library handles:

- SHA-2 compression function implementation (64/80 rounds)
- Message scheduling and padding operations
- Key derivation and HMAC two-round processing
- All internal cryptographic transformations

This architectural separation ensures:

- Proven cryptographic correctness through established Botan library
 - Maintainability by separating concerns (orchestration vs. computation)
 - Appropriate abstraction level for virtual platform modeling
 - Focus on software-visible functional behavior
-

2. Design Overview

2.1 Functional Description

The HMAC peripheral provides hardware-accelerated cryptographic hashing and message authentication services supporting:

- **Standalone SHA-2 hashing:** Compute secure hash digests without authentication
- **HMAC mode:** Generate keyed message authentication codes using SHA-2 as the underlying hash function

The peripheral supports three digest sizes:

- **SHA-2 256:** 256-bit digest output (32 bytes)
- **SHA-2 384:** 384-bit digest output (48 bytes)
- **SHA-2 512:** 512-bit digest output (64 bytes)

Key capabilities include:

- **Flexible message input:** Memory-mapped FIFO interface with 4KB address window supporting burst writes and DMA transfers
- **Configurable key lengths:** Support for 128-bit, 256-bit, 384-bit, 512-bit, and 1024-bit secret keys

- **Context switching:** Ability to save and restore hash state for time-interleaved processing of multiple message streams
- **Error detection:** Comprehensive validation of software command sequences and configuration parameters
- **Security features:** Write-only key registers and secret wipe capability

2.2 Operating Modes

2.2.1 SHA-2 Mode In SHA-2 mode (`CFG.hmac_en = 0`), the peripheral computes standard SHA-2 hash digests:

1. Software configures digest size via `CFG.digest_size`
2. Software enables SHA engine via `CFG.sha_en`
3. Software issues `hash_start` command via `CMD` register
4. Software writes message data to `MSG_FIFO` window (0x1000-0x1FFF)
5. Software issues `hash_process` command to finalize computation
6. Hardware sets `INTR_STATE.hmac_done` interrupt when digest is ready
7. Software reads digest output from `DIGEST_0` to `DIGEST_7/11/15` depending on digest size

2.2.2 HMAC Mode In HMAC mode (`CFG.hmac_en = 1`), the peripheral computes keyed message authentication codes:

1. Software configures digest size and key length via `CFG` register
2. Software writes secret key to `KEY_0` to `KEY_31` registers (number depends on key length)
3. Software enables SHA engine and HMAC mode via `CFG` register
4. Software issues `hash_start` command via `CMD` register
5. Hardware performs inner hash computation: $H(\text{key XOR ipad} \parallel \text{message})$
6. Software writes message data to `MSG_FIFO` window
7. Software issues `hash_process` command to finalize computation
8. Hardware performs outer hash computation: $H(\text{key XOR opad} \parallel \text{inner_hash})$
9. Hardware sets `INTR_STATE.hmac_done` interrupt when HMAC is ready
10. Software reads HMAC output from `DIGEST` registers

HMAC mode adds 240 additional cycles to completion time (80 cycles for inner key processing, 80 cycles for outer key processing, 80 cycles for first-round result processing).

2.3 Performance Characteristics

Block processing latencies are software-observable and affect throughput estimation:

- **SHA-2 256:** 80 cycles per 512-bit block (64 bytes / 80 cycles)
- **SHA-2 384/512:** 96 cycles per 1024-bit block (128 bytes / 96 cycles)
- **HMAC overhead:** Additional 240 cycles for two-round computation

At 100 MHz clock frequency:

- **SHA-2 256 throughput:** ~80 MB/s
- **SHA-2 384/512 throughput:** ~133 MB/s
- **HMAC throughput:** Reduced by two-round overhead plus key processing time

These timing characteristics are modeled as TLM timing annotations to provide accurate performance estimates for software development.

3. Features

3.1 TLM-Relevant Features (Is)

The following features are modeled in the SystemC TLM implementation as they represent software-visible functional behavior:

3.1.1 Operational Modes

- Dual-mode operation supporting standalone SHA-2 hashing and keyed HMAC authentication
- Multi-mode SHA-2 engine supporting SHA-2 256, SHA-2 384, and SHA-2 512 digest sizes
- Two-round HMAC computation with inner and outer key padding orchestration

3.1.2 Message Processing

- Memory-mapped message FIFO interface with 4KB address window for flexible write access
- Support for arbitrary message lengths with automatic padding coordination to block boundaries
- Message byte-order conversion between little-endian and big-endian formats
- Message FIFO depth management (16 entries for SHA-2 256, 32 entries for SHA-2 384/512)
- FIFO full condition causing interconnect back-pressure when software writes to full FIFO

3.1.3 Key Management

- Configurable secret key lengths: 128-bit, 256-bit, 384-bit, 512-bit, and 1024-bit
- Key length constraints: maximum 512-bit for SHA-2 256, maximum 1024-bit for SHA-2 384/512
- Software-configurable key byte-order swapping
- Key register write protection during active hash computation

3.1.4 Configuration and Control

- Software-controlled hash operation commands: start, stop, continue, and process
- Configurable digest output byte-order swapping
- Dynamic configuration updates restricted to idle state
- Configuration validation with error reporting for invalid digest size and key length combinations

3.1.5 Context Switching

- Support for time-interleaved processing of multiple message streams
- Context save and restore capability at message block boundaries
- Preservation of digest state and message length counters across context switches

3.1.6 Interrupt Generation

- Hash completion interrupt indicating digest availability
- Message FIFO empty interrupt with conditional triggering based on prior full condition
- Error interrupt for reporting configuration and operational violations

3.1.7 Error Detection and Reporting

- Software error detection for writes to message FIFO when SHA engine disabled
- Error reporting for hash start command when SHA engine disabled
- Detection of secret key register updates during active hashing
- Error signaling for hash start command while engine already active
- Detection of message FIFO writes after hash process command issued
- Configuration validation errors for invalid mode combinations

3.1.8 Status Monitoring

- FIFO status indicators: full, empty, and depth reporting
- Engine idle status for detecting completion of hash operations
- Message length tracking in bits with 64-bit counter

3.1.9 Security Features

- Secret key wipe capability to clear internal key storage
- Write-only key registers preventing software readback of secret keys
- Fatal alert generation on TL-UL bus integrity violations

3.1.10 Data Transfer

- Wide address range MSG_FIFO window enabling efficient burst writes and store-multiple instructions
- Support for sub-word writes with automatic packing into 32-bit words
- DMA-friendly interface supporting fixed write address with sequential data streaming

3.1.11 Performance Characteristics

- Block processing latency observable to software: 80 cycles per 512-bit block for SHA-2 256
- Block processing latency observable to software: 96 cycles per 1024-bit block for SHA-2 384/512
- Additional HMAC overhead observable to software: 240 cycles for two-round keyed hash computation
- Timing characteristics affecting software throughput estimation and performance tuning

3.2 Excluded Features (Is Not)

The following features are NOT modeled in the SystemC TLM implementation as they represent RTL/analog/physical characteristics or implementation details below the transaction abstraction level:

3.2.1 Cryptographic Algorithm Implementation Details

- Internal SHA-2 compression function implementation with 64/80 rounds (Botan library handles cryptographic computation)
- Message scheduling array word derivation and shuffling operations (Botan library responsibility)
- Round constant application and intermediate hash computations (Botan library responsibility)
- Bit-level padding insertion and message length encoding logic (abstracted interface to Botan)
- Internal state rotation and transformation operations within compression function (Botan library responsibility)

3.2.2 Implementation Details

- Internal message scheduling FIFO array micro-architecture (not software-visible)
- Hash register and digest register internal organization
- Packer logic internal implementation for message word coalescing
- Register file banking and partitioning strategies

3.2.3 Physical and Electrical Characteristics

- Voltage levels and current consumption specifications

- Power domain isolation and voltage scaling requirements
- Temperature operating ranges and thermal characteristics
- Process corner variations and manufacturing tolerances

3.2.4 Timing at Pin Level

- Setup and hold times for input signals
- Clock-to-output delays for interface signals
- Propagation delays through combinational logic paths
- Cycle-accurate compression round progression below observable block level

3.2.5 Test and Debug Features

- Design-for-test scan chain insertion
- JTAG boundary scan implementation
- Manufacturing test modes and built-in self-test patterns
- Debug access port functionality for silicon validation

3.2.6 Gate-Level Implementation

- Logic gate composition and transistor-level design
- Standard cell library utilization and optimization
- Clock tree synthesis and distribution network
- Place and route floorplanning details

3.2.7 Analog Behavior

- Clock jitter and phase noise characteristics
- Signal integrity and crosstalk effects
- Power supply noise and ground bounce phenomena
- Electromagnetic interference susceptibility

3.2.8 Unimplemented Features

- Future performance optimization for pipelined message feeding during compression rounds
- Reserved register fields with no current functional purpose
- Placeholder functionality for forward compatibility

3.2.9 Hardware-Only Optimizations

- Internal pipeline staging between datapath elements (below transaction level granularity)
- Clock gating insertion for power reduction (not observable to software)
- Datapath width optimizations for area-power tradeoffs (not affecting functional behavior)

4. Configuration Parameters

4.1 Overview

The HMAC SystemC TLM model requires 8 build-time configuration parameters that are fixed at IP synthesis/compilation time. These parameters define hardware resources, timing characteristics, and register array sizes that affect the functional behavior and structure of the TLM model.

Important Context: The SystemC model orchestrates data flow, register access, and status management while delegating actual cryptographic operations to the Botan library. These parameters define the hardware resources and timing characteristics that must be modeled.

4.2 Configuration Parameter Table

Parameter Name	Type	Default Value	Description	Source
MSG_FIFO_DEPTH_HMAC_SHA256	int	16 entries	Message FIFO depth for SHA-2 256 mode. Fixed hardware resource determining FIFO full/empty status generation and back-pressure behavior. Model tracks FIFO occupancy against this limit to generate correct STATUS.fifo_full and STATUS.fifo_empty flags.	theory_of_operation.md

Parameter Name	Type	Default Value	Description	Source
MSG_FIFO_DEPTH	int	SHA2 384/512 tries	Message FIFO depth for SHA-2 384/512 modes. Fixed hardware resource for FIFO status generation. Model dynamically uses this depth based on configured digest size.	theory_of_operation.md
MSG_FIFO_WINDOW_SIZE	int	4096 bytes (0x1000)	Address window size for MSG_FIFO memory-mapped region. Affects TLM address decoder and determines valid address range (0x1000 to 0x1FFF) for message FIFO writes. Any write within this window directed to message FIFO.	registers.md

Parameter Name	Type	Default Value	Description	Source
SHA256_BLOCK_CYCLES	unsigned int	80 cycles	Processing latency for one 512-bit message block in SHA-2 256 mode. Observable timing characteristic affecting software throughput estimation. Model introduces this delay when processing each complete 512-bit block.	theory_of_operation.md
SHA384_512_BLOCK_CYCLES	unsigned int	96 cycles	Processing latency for one 1024-bit message block in SHA-2 384/512 modes. Observable timing characteristic for transaction timing. Model uses this delay when digest size configured for SHA-2 384 or SHA-2 512.	theory_of_operation.md

Parameter Name	Type	Default Value	Description	Source
HMAC_EXTRA_CYCLES	unsigned int	240 cycles	Additional latency overhead for HMAC mode due to two-round computation (inner and outer key processing). Includes 80 cycles for inner key, 80 cycles for outer key, and 80 cycles for processing first round result. Added to total hash completion time when HMAC mode enabled.	theory_of_operation.md
NUM_DIGEST_REGS	unsigned int	16 registers	Number of 32-bit DIGEST registers (DIGEST_0 to DIGEST_15). Defines size of digest register array. Different digest sizes use different subsets: SHA-2 256 uses 0-7 (256 bits), SHA-2 384 uses 0-11 (384 bits), SHA-2 512 uses 0-15 (512 bits).	registers.md, programmers_guide.md

Parameter Name	Type	Default Value	Description	Source
NUM_KEY_REGS	unsigned int	32 registers	Number of 32-bit KEY registers (KEY_0 to KEY_31). Defines size of secret key register array. Maximum key length is 1024 bits (32 x 32-bit words). Number of relevant registers depends on configured key length at runtime.	registers.md

4.3 Parameter Usage Notes

4.3.1 FIFO Depth Selection The model must dynamically select between MSG_FIFO_DEPTH_SHA256 (16 entries) and MSG_FIFO_DEPTH_SHA384_512 (32 entries) based on the runtime configuration of CFG.digest_size. However, both depth values are build-time constants that define the hardware capabilities. The larger FIFO depth for SHA-2 384/512 modes accommodates the larger block size (1024 bits vs. 512 bits).

4.3.2 Timing Parameters The cycle count parameters (SHA256_BLOCK_CYCLES, SHA384_512_BLOCK_CYCLES, HMAC_EXTRA_CYCLES) are used for TLM timing annotations and should be exposed to testbenches for calculating expected completion times. These represent observable hardware characteristics that software can use for throughput estimation and performance tuning.

4.3.3 Register Array Sizes NUM_DIGEST_REGS and NUM_KEY_REGS define the physical register map structure. While runtime configuration determines how many registers are actually relevant for a given operation (e.g., SHA-2 256 only uses DIGEST_0 to DIGEST_7), the full register set must be instantiated in the model to match the hardware interface.

4.3.4 No Mode Selection Parameters This IP does not have build-time parameters to disable specific digest modes or HMAC functionality. All modes

(SHA-2 256/384/512, HMAC enable/disable) are runtime-selectable via the CFG register, which is part of the software-programmable interface. This provides maximum flexibility for software without requiring hardware reconfiguration.

4.4 Excluded Runtime-Configurable Features

The following are NOT build-time configuration parameters as they are runtime-programmable via registers:

- **Digest size selection** (SHA-2 256/384/512) - Configured via CFG.digest_size register
- **Key length selection** (128/256/384/512/1024 bits) - Configured via CFG.key_length register
- **HMAC mode enable/disable** - Configured via CFG.hmac_en register
- **SHA engine enable** - Configured via CFG.sha_en register
- **Byte-order swapping** (endian_swap, digest_swap, key_swap) - Configured via CFG register fields
- **Interrupt enable masks** - Configured via INTR_ENABLE register
- **Hash operation commands** (start/stop/continue/process) - Issued via CMD register

These runtime features are handled through the standard TLM register read/write interface and do not require build-time parameterization.

5. Port Interfaces

5.1 Overview

The HMAC SystemC TLM model exposes 6 interface groups for integration into virtual platforms. These interfaces provide register access, interrupt signaling, security alerting, clock input, and reset control.

5.2 Port Interface Table

Interface Category	Port Name	Port Type	Description
Register Bus	tl_socket	t1m_target_socket	TL-32 register bus interface for memory-mapped register access including CFG, CMD, STATUS, ERR_CODE, KEY_, <i>DIGEST_</i> , MSG_LENGTH_*, and MSG_FIFO window (0x0-0x1FFF)
Interrupt	intr_hmac_donesc_out		Hash completion interrupt - asserted when HMAC/SHA-2 operation completes and digest is ready to be read
Interrupt	intr_fifo_empty	sc_out	Message FIFO empty interrupt - asserted when FIFO becomes empty after being full, indicating software can write more message data
Interrupt	intr_hmac_err	sc_out	Error interrupt - asserted when configuration or operational errors occur (invalid commands, key updates during hashing, FIFO writes after process command)

Interface Category	Port Name	Port Type	Description
Alert	alert_fatal_faultsc_out		Fatal security alert - asserted on TL-UL bus integrity violations detected by end-to-end integrity checking
Clock	clk_i	sc_in	Functional clock input representing clock frequency in Hz for timing annotation of block processing latencies
Reset	rst_ni	sc_in	Active-low asynchronous reset input for initializing module state

5.3 Interface Details

5.3.1 Register Bus Interface **Port:** tl_socket (tlm_target_socket<32>)

The TL-UL register bus interface provides memory-mapped access to all configuration, control, status, and data registers. The interface supports:

- **32-bit data width:** All registers are 32-bit aligned
- **TLM2.0 compliance:** Standard blocking transport interface
- **Address range:** 0x0000 to 0x1FFF (8KB total)
 - Configuration and status registers: 0x0000 to 0x0FFF
 - Message FIFO window: 0x1000 to 0x1FFF (4KB window)

The register bus provides access to:

- **Configuration registers:** CFG (mode, digest size, key length, byte-order control)
- **Command register:** CMD (hash_start, hash_process, hash_stop, hash_continue)
- **Status registers:** STATUS (idle, FIFO status), ERR_CODE (error details)
- **Interrupt registers:** INTR_STATE, INTR_ENABLE, INTR_TEST
- **Alert register:** ALERT_TEST

- **Security register:** WIPE_SECRET
- **Key registers:** KEY_0 to KEY_31 (32 write-only registers for secret key input)
- **Digest registers:** DIGEST_0 to DIGEST_15 (16 read/write registers for hash output)
- **Message length registers:** MSG_LENGTH_LOWER, MSG_LENGTH_UPPER
- **Message FIFO:** MSG_FIFO window (0x1000-0x1FFF) for message data input

5.3.2 Interrupt Outputs Ports: intr_hmac_done, intr_fifo_empty, intr_hmac_err (sc_out)

Three interrupt output signals provide asynchronous notification of completion and error conditions:

intr_hmac_done: - Asserted when hash or HMAC computation completes - Indicates digest is ready to be read from DIGEST registers - Cleared by writing 1 to INTR_STATE.hmac_done (W1C) - Maskable via INTR_ENABLE.hmac_done

intr_fifo_empty: - Asserted when message FIFO becomes empty - Conditional assertion: only triggered after FIFO was previously full - Used for DMA coordination and flow control - Automatically cleared when FIFO becomes non-empty - Maskable via INTR_ENABLE.fifo_empty (though status is read-only)

intr_hmac_err: - Asserted on configuration or operational errors - Error types include: - Hash start when SHA engine disabled - Hash start when engine already active - Secret key update during active hashing - Message FIFO write when operation not allowed - Invalid configuration (digest size, key length combinations) - Error details available in ERR_CODE register - Cleared by writing 1 to INTR_STATE.hmac_err (W1C) - Maskable via INTR_ENABLE.hmac_err

All interrupts are masked by corresponding bits in INTR_ENABLE register. Output signal state = INTR_STATE & INTR_ENABLE (logical AND).

5.3.3 Security Alert Port: alert_fatal_fault (sc_out)

Fatal security alert output for critical system-level violations:

- Asserted on TL-UL bus integrity violations
- Detected by end-to-end integrity checking mechanisms
- Requires system-level response (may trigger system reset or security lockdown)
- Testable via ALERT_TEST.fatal_fault register write

5.3.4 Clock Input Port: clk_i (sc_in)

Functional clock input for timing annotations:

- Represents clock frequency in Hz
- Used to calculate absolute time delays for block processing

- Timing delays:
 - SHA-2 256: 80 clock cycles per 512-bit block
 - SHA-2 384/512: 96 clock cycles per 1024-bit block
 - HMAC overhead: additional 240 cycles
- Clock input used for TLM timing annotations (wait statements)

Note: SystemC TLM model does not implement cycle-accurate behavior. Clock input is used solely for annotating transaction-level timing delays.

5.3.5 Reset Input **Port:** rst_ni (sc_in)

Active-low asynchronous reset for module initialization:

- Active-low polarity (assert by driving to 0)
- Asynchronous operation (can assert at any time)
- Resets all registers to default values
- Clears internal state machines
- Initializes FIFO to empty state
- De-asserts all interrupt and alert outputs

Reset behavior:

- All configuration registers reset to default values (CFG = 0x4100)
- All key and digest registers cleared to 0x0
- Command register cleared
- Interrupt state cleared
- FIFO emptied
- State machine returned to IDLE

6. Memory-Mapped Registers

6.1 Overview

The HMAC peripheral provides 59 registers/register groups organized into 9 functional categories for software control and data transfer. All registers are 32-bit aligned and accessed through the TL-UL register bus interface.

Architectural Context: The SystemC TLM model orchestrates data flow, register access, and status management while delegating actual cryptographic computations to the Botan library. These registers provide the software-visible interface for HMAC/SHA-2 operations.

6.2 Register Map Summary

Address Range	Register Group	Count	Access	Purpose
0x00 - 0x08	Interrupt Control	3	Mixed	Interrupt status, enable, and test
0x0C	Alert Test	1	WO	Fatal fault alert injection
0x10 - 0x14	Configuration & Control	2	Mixed	Mode configuration and command interface
0x18 - 0x1C	Status & Error	2	RO	Engine status and error code reporting
0x20	Security	1	WO	Secret key wipe
0x24 - 0xA0	Key Input	32	WO	Secret key registers (1024-bit total)
0xA4 - 0xE0	Digest Output	16	RW	Hash/HMAC result registers (512-bit total)
0xE4 - 0xE8	Message Length	2	RW	Message length tracking (64-bit counter)
0x1000 - 0x1FFF	Message FIFO	1 window	WO	Message data input (4KB address window)

Total Registers: 59 registers/register groups

6.3 Interrupt Registers

Register Name	Offset	Size	Access	Reset	Description
INTR_STATE	0x00	32-bit	Mixed	0x0	Interrupt state register with three interrupt sources: hmac_done (bit 0, rw1c), fifo_empty (bit 1, ro), hmac_err (bit 2, rw1c). Write 1 to clear hmac_done and hmac_err.
INTR_ENABLE	0x04	32-bit	RW	0x0	Interrupt enable register for masking interrupt sources: hmac_done (bit 0), fifo_empty (bit 1), hmac_err (bit 2). Output = INTR_STATE & INTR_ENABLE.
INTR_TEST	0x08	32-bit	WO	0x0	Interrupt test register for software interrupt injection: hmac_done (bit 0), fifo_empty (bit 1), hmac_err (bit 2). Writing 1 forces interrupt assertion.

Interrupt Sources:

- **hmac_done (bit 0):** Hash/HMAC computation completed, digest ready

- **fifo_empty (bit 1)**: Message FIFO empty after being full
- **hmac_err (bit 2)**: Configuration or operational error occurred

6.4 Alert Register

Register Name	Offset	Size	Access	Reset	Description
ALERT_TEST	0x0C	32-bit	WO	0x0	Alert test register for TL-UL bus integrity violation testing: fatal_fault (bit 0). Writing 1 triggers alert_fatal_fault output.

6.5 Configuration and Control Registers

Register Name	Offset	Size	Access	Reset	Description
CFG	0x10	32-bit	RW	0x4100	Configuration register with fields: hmac_en (bit 0), sha_en (bit 1), endian_swap (bit 2), digest_swap (bit 3), key_swap (bit 4), digest_size (bits 8:5, one-hot), key_length (bits 14:9, one-hot). Updated only when engine idle.

Register Name	Offset	Size	Access	Reset	Description
CMD	0x14	32-bit	Mixed	0x0	Command register with self-clearing fields: hash_start (bit 0, r0w1c), hash_process (bit 1, r0w1c), hash_stop (bit 2, r0w1c), hash_continue (bit 3, r0w1c). Commands trigger state transitions.

CFG Register Fields:

- **hmac_en (bit 0)**: Enable HMAC mode (0 = SHA-2 only, 1 = HMAC)
- **sha_en (bit 1)**: Enable SHA engine (must be 1 for operation)
- **endian_swap (bit 2)**: Swap message data byte order
- **digest_swap (bit 3)**: Swap digest output byte order
- **key_swap (bit 4)**: Swap key input byte order
- **digest_size (bits 8:5)**: One-hot encoded (bit 5 = SHA-2 256, bit 6 = SHA-2 384, bit 7 = SHA-2 512, bit 8 = reserved)
- **key_length (bits 14:9)**: One-hot encoded (bit 9 = 128-bit, bit 10 = 256-bit, bit 11 = 384-bit, bit 12 = 512-bit, bit 13 = 1024-bit, bit 14 = reserved)

CMD Register Commands:

- **hash_start (bit 0)**: Start new hash operation (validates configuration)
- **hash_process (bit 1)**: Finalize current message and compute digest
- **hash_stop (bit 2)**: Stop after current block and save context
- **hash_continue (bit 3)**: Resume hashing from saved context

6.6 Status and Error Registers

Register Name	Offset	Size	Access	Reset	Description
STATUS	0x18	32-bit	RO	0x3	Status register with fields: hmac_idle (bit 0), fifo_empty (bit 1), fifo_full (bit 2), fifo_depth (bits 9:4). Live hardware status reflection.
ERR_CODE	0x1C	32-bit	RO	0x0	Error code register providing detailed error information when hmac_err interrupt occurs. Error codes: 0x1 (SwHash-StartWhen-ShaDisabled), 0x2 (reserved), 0x3 (SwUpdateSecretKey-InProcess), 0x4 (SwHash-StartWhenActive), 0x5 (SwPushMsg-WhenDisallowed), 0x6 (SwInvalidConfig).

STATUS Register Fields:

- **hmac_idle (bit 0):** Engine idle, context-switch registers writable
- **fifo_empty (bit 1):** Message FIFO empty
- **fifo_full (bit 2):** Message FIFO full (software should wait)
- **fifo_depth (bits 9:4):** Current FIFO occupancy (0 to 16/32 depending on mode)

ERR_CODE Values:

- **0x1 (SwHashStartWhenShaDisabled)**: Hash start issued when SHA engine disabled
- **0x3 (SwUpdateSecretKeyInProcess)**: Key register write during active hashing
- **0x4 (SwHashStartWhenActive)**: Hash start issued when engine already active
- **0x5 (SwPushMsgWhenDisallowed)**: Message FIFO write when operation not allowed
- **0x6 (SwInvalidConfig)**: Invalid digest size or key length configuration

6.7 Security Register

Register Name	Offset	Size	Access	Reset	Description
WIPE_SECRET	0x20	32-bit	WO	0x0	Secret wipe register for clearing internal key storage, intermediate hash results, digest, and message scheduling array. Software writes 32-bit value from entropy source to overwrite security-sensitive data.

6.8 Key Registers

32 write-only registers for secret key input (KEY_0 to KEY_31):

Register Range	Offset Range	Size Each	Access	Reset	Description
KEY_0 to KEY_31	0x24 - 0xA0	32-bit	WO	0x0	Secret key register array (1024-bit total capacity). Write-only for security. Write- protected during active hashing. Key array ordering: key[1023:0] = {KEY_0, KEY_1, ..., KEY_31}.

Key Length Usage:

- **128-bit key:** Use KEY_0 to KEY_3 (4 registers)
- **256-bit key:** Use KEY_0 to KEY_7 (8 registers)
- **384-bit key:** Use KEY_0 to KEY_11 (12 registers)
- **512-bit key:** Use KEY_0 to KEY_15 (16 registers)
- **1024-bit key:** Use KEY_0 to KEY_31 (32 registers)

Security Notes:

- All key registers are write-only
- Software cannot read back key values
- Write attempts during active hashing rejected with ERR_CODE = 0x3
- Key registers cleared by WIPE_SECRET operation

6.9 Digest Output Registers

16 read/write registers for hash/HMAC result output (DIGEST_0 to DIGEST_15):

Register Range	Offset Range	Size Each	Access	Reset	Description
DIGEST_0 to DIGEST_15	0xA4 - 0xE0	32-bit	RW	0x0	Digest output register array (512-bit total capacity). Writable when hmac_idle=1 for context switching. Digest ordering: digest[511:0] = {DI- GEST_0, DIGEST_1, ..., DIGEST_15}.

Digest Size Usage:

- **SHA-2 256:** Read DIGEST_0 to DIGEST_7 (8 registers, 256 bits)
- **SHA-2 384:** Read DIGEST_0 to DIGEST_11 (12 registers, 384 bits)
- **SHA-2 512:** Read DIGEST_0 to DIGEST_15 (16 registers, 512 bits)

Context Switching:

- DIGEST registers are writable when STATUS.hmac_idle = 1
- Used to save and restore hash state for time-interleaved processing
- Write attempts during active hashing rejected

6.10 Message Length Registers

Register Name	Offset	Size	Access	Reset	Description
MSG_LENGTH_LOWER	0x04	32-bit	RW	0x0	Message length lower 32 bits [31:0] in bits (byte granularity, lower 3 bits ignored). Writable when hmac_idle=1 for context switching.
MSG_LENGTH_UPPER	0x08	32-bit	RW	0x0	Message length upper 32 bits [63:32] in bits. Writable when hmac_idle=1. For SHA-2 256: 64-bit length = {MSG_LENGTH_UPPER, MSG_LENGTH_LOWER}. For SHA-2 384/512: 128-bit length = {32'b0, 32'b0, MSG_LENGTH_UPPER, MSG_LENGTH_LOWER}.

Message Length Format:

- **Unit:** Bits (not bytes)
- **Granularity:** Byte-aligned (lower 3 bits should be 0)
- **Width:** 64-bit counter (sufficient for 2^{64} bits = 2^{61} bytes approximately 2 exabytes)
- **Context Switching:** Writable when idle for state save/restore

6.11 Message FIFO Register

Register Name	Offset	Size	Access	Reset	Description
MSG_FIFO	0x1000 - 0x1FFF	32-bit	WO	N/A	Message FIFO write window (4KB address range). Any write to this address range appends data to message FIFO. Supports byte, half-word, and word writes. Hardware handles 4KB window mapping internally. DMA-friendly interface with back-pressure on FIFO full condition.

MSG_FIFO Characteristics:

- **Address Window:** 4096 bytes (0x1000 to 0x1FFF)
- **Address Mapping:** All addresses within window map to same FIFO entry point
- **Sub-Word Writes:** Byte (8-bit), half-word (16-bit), and word (32-bit) writes supported
- **Write Alignment:** Lower 2 address bits used for sub-word alignment
- **DMA Support:** Fixed write address with sequential data streaming
- **Back-Pressure:** TLM transaction blocked when FIFO full (STAUTUS.fifo_full = 1)
- **Write Protection:** Writes rejected with error when:
 - SHA engine disabled (CFG.sha_en = 0)
 - Hash operation not started (no hash_start or hash_continue)
 - Hash already finalized (hash_process or hash_stop issued)

6.12 Access Type Legend

- **RO:** Read Only
- **RW:** Read/Write
- **WO:** Write Only

- **Mixed:** Contains fields with different access types
- **W1C:** Write-1-to-Clear (writing 1 clears bit, writing 0 has no effect)
- **r0w1c:** Read-0-Write-1-to-Clear (reads as 0, writing 1 triggers action and clears bit)

6.13 Key Architectural Notes

1. **Configuration Protection:** CFG register can only be updated when STATUS.hmac_idle = 1
 2. **Key Security:** KEY registers are write-only and protected during active hashing
 3. **Context Switching:** DIGEST and MSG_LENGTH registers writable only when idle
 4. **FIFO Window:** Wide address range enables efficient DMA transfers and burst writes
 5. **Error Detection:** Comprehensive validation with ERR_CODE details for all errors
 6. **Botan Integration:** Registers manage orchestration; Botan library handles cryptographic computation
-

7. Register Callbacks

7.1 Overview

Register callbacks are functions that execute immediately when specific registers are read or written. These callbacks implement functional side-effects such as state transitions, access control validation, interrupt generation, and data movement. The HMAC peripheral requires 60 register callbacks across 12 functional groups.

Architectural Context: The SystemC TLM model orchestrates data flow, register access, and status management while delegating cryptographic computations to the Botan library. Callbacks handle all orchestration behaviors, access control, and hardware side-effects.

7.2 Callback Definition Criteria

A register requires a callback if it meets either criterion:

Immediate Functional Side-Effects:

- State or mode changes (enables/disables logic, starts/stops operations)
- FIFO/data movement (pushes/pops data, advances pointers)
- Event/flag clearing (W1C or read-to-clear status/error flags)
- Triggering outputs (asserts/de-asserts interrupt or alert ports)
- Access control enforcement (rejects/modifies writes based on state)
- Protocol actions (initiates hash operations, processes blocks)

Conditional Access Dependencies: - Write protection (register writable only when specific conditions met) - Mode-dependent access (behavior changes based on configuration) - Lock mechanisms (access blocked when engine active) - Enable dependencies (functionality only active when enables set)

7.3 Callback Summary by Category

Category	Callbacks	Description
Interrupt Control	3	INTR_STATE (write), INTR_ENABLE (write), INTR_TEST (write)
Alert Test	1	ALERT_TEST (write)
Configuration	2	CFG (read + write)
Command	1	CMD (write)
Security	1	WIPE_SECRET (write)
Key Input	32	KEY_0 to KEY_31 (write each)
Digest Output	32	DIGEST_0 to DIGEST_15 (read + write each)
Message Length	2	MSG_LENGTH_LOWER (write), MSG_LENGTH_UPPER (write)
Message FIFO	1	MSG_FIFO (write)
Status	1	STATUS (read)
Total	60	

7.4 Detailed Callback Specifications

7.4.1 Interrupt Control Callbacks **handle_write_INTR_STATE:** - **Type:** Write callback - **Purpose:** Clear interrupt flags using W1C semantics - **Behavior:** - Writing 1 to hmac_done (bit 0) clears interrupt flag - Writing 1 to hmac_err (bit 2) clears interrupt flag - fifo_empty (bit 1) is read-only, cannot be cleared by software - Update intr_hmac_done and intr_hmac_err output ports based on new state and INTR_ENABLE mask - Output = INTR_STATE & INTR_ENABLE

handle_write_INTR_ENABLE: - **Type:** Write callback - **Purpose:** Update interrupt mask register - **Behavior:** - Store new interrupt enable mask - Immediately re-evaluate and update all interrupt output signals - intr_hmac_done = INTR_STATE.hmac_done & INTR_ENABLE.hmac_done - intr_fifo_empty = INTR_STATE.fifo_empty & INTR_ENABLE.fifo_empty - intr_hmac_err = INTR_STATE.hmac_err & INTR_ENABLE.hmac_err

handle_write_INTR_TEST: - **Type:** Write callback - **Purpose:** Force interrupt assertion for testing - **Behavior:** - Writing 1 to any bit sets corre-

sponding INTR_STATE bit - Update interrupt output ports based on new state and INTR_ENABLE mask - Used for software interrupt testing and verification

7.4.2 Alert Test Callback handle_write_ALERT_TEST: - **Type:** Write callback - **Purpose:** Force fatal fault alert assertion for testing - **Behavior:** - Writing 1 to fatal_fault (bit 0) triggers alert_fatal_fault output port - Used for testing alert propagation and system response

7.4.3 Configuration Callbacks handle_read_CFG: - **Type:** Read callback - **Purpose:** Return current configuration - **Behavior:** - Return current CFG register value - Note: Software should maintain separate copy for context switching since reading during active operation may not reflect applied values

handle_write_CFG: - **Type:** Write callback - **Purpose:** Validate and update configuration - **Behavior:** - Validate write occurs only when hmac_idle = 1 (STATUS.hmac_idle) - If engine active, discard write (no error reported per specification) - If engine idle, validate configuration: - Reject digest_size = SHA2_None (value 0x8) - For HMAC mode: reject key_length = Key_None (value 0x20) - For SHA-2 256 mode: reject key_length = Key_1024 (value 0x2000) - If validation fails: - Set ERR_CODE to SwInvalidConfig (0x6) - Set INTR_STATE.hmac_err and assert intr_hmac_err - If validation succeeds: - Update CFG register - Update Botan library setup parameters

7.4.4 Command Callback handle_write_CMD: - **Type:** Write callback - **Purpose:** Process hash operation commands - **Behavior:**

hash_start (bit 0): - Validate sha_en = 1 in CFG register - If disabled: Set ERR_CODE to SwHashStartWhenShaDisabled (0x1), assert intr_hmac_err - Validate engine not already active - If active: Set ERR_CODE to SwHashStartWhenActive (0x4), assert intr_hmac_err - Validate digest_size and key_length configuration - If invalid: Set ERR_CODE to SwInvalidConfig (0x6), assert intr_hmac_err - If validation succeeds: - Transition state from IDLE to PROCESSING - Initialize Botan library hash/HMAC context - Clear digest registers - Clear message length counter - Clear command bit (self-clearing)

hash_process (bit 1): - Finalize current message processing - Apply message padding (delegated to Botan) - Compute final hash digest (delegated to Botan) - For HMAC mode: Perform outer hash computation (delegated to Botan) - Transition to completion state - Write digest result to DIGEST registers - Set INTR_STATE.hmac_done and assert intr_hmac_done - Clear command bit (self-clearing) - Annotate timing delay: block cycles + HMAC overhead if applicable

hash_stop (bit 2): - Stop after current block completes - Save context for switching (digest state and message length) - Set INTR_STATE.hmac_done and assert intr_hmac_done - Transition state to IDLE - Clear command bit (self-clearing)

hash_continue (bit 3): - Validate hmac_idle = 1 in STATUS register - Validate DIGEST and MSG_LENGTH registers have been restored - Resume hashing from saved context - Restore Botan library state from DIGEST registers - Transition state from IDLE to PROCESSING - Clear command bit (self-clearing)

7.4.5 Security Callback handle_write_WIPE_SECRET: - **Type:** Write callback - **Purpose:** Clear security-sensitive data - **Behavior:** - Overwrite internal key storage with written 32-bit value - Overwrite intermediate hash results with written value - Overwrite digest registers with written value - Overwrite message scheduling array with written value - For SHA-2 384/512: Duplicate 32-bit value to create 64-bit wipe pattern - Used after operations to prevent key recovery

7.4.6 Key Input Callbacks (32 callbacks) handle_write_KEY_0 to handle_write_KEY_31: - **Type:** Write callbacks (one for each KEY register) - **Purpose:** Store secret key with write protection - **Behavior:** - Validate write occurs only when engine idle (hmac_idle = 1) - If engine active (PROCESSING state): - Reject write - Set ERR_CODE to SwUpdateSecretKeyInProgress (0x3) - Set INTR_STATE.hmac_err and assert intr_hmac_err - If engine idle: - Store key word in internal key register array - Key available for Botan HMAC context initialization on hash_start

7.4.7 Digest Output Callbacks (32 callbacks) handle_read_DIGEST_0 to handle_read_DIGEST_15 (16 read callbacks): - **Type:** Read callbacks (one for each DIGEST register) - **Purpose:** Return computed hash/HMAC result - **Behavior:** - Return computed digest word from Botan library result - Digest availability indicated by INTR_STATE.hmac_done - Different digest sizes use different register counts: - SHA-2 256: Read DIGEST_0 to DIGEST_7 - SHA-2 384: Read DIGEST_0 to DIGEST_11 - SHA-2 512: Read all DIGEST_0 to DIGEST_15

handle_write_DIGEST_0 to handle_write_DIGEST_15 (16 write callbacks): - **Type:** Write callbacks (one for each DIGEST register) - **Purpose:** Support context switching via state save/restore - **Behavior:** - Validate write occurs only when hmac_idle = 1 (STATUS.hmac_idle) - If engine active: - Reject write (would cause unpredictable behavior) - If engine idle: - Store digest word for context switching/restoration - Used when resuming hashing via hash_continue command - Botan library state restored from these registers

7.4.8 Message Length Callbacks handle_write_MSG_LENGTH_LOWER: - **Type:** Write callback - **Purpose:** Store message length lower 32 bits for context switching - **Behavior:** - Validate write occurs only when hmac_idle = 1 (STATUS.hmac_idle) - If engine active: Reject write (unpredictable behavior) - If engine idle: Store message length lower 32 bits for context switching

handle_write_MSG_LENGTH_UPPER: - **Type:** Write callback - **Purpose:** Store message length upper 32 bits for context switching - **Behavior:** - Validate write occurs only when `hmac_idle = 1` - If engine active: Reject write - If engine idle: Store message length upper 32 bits for context switching

7.4.9 Message FIFO Callback **handle_write_MSG_FIFO:** - **Type:** Write callback - **Purpose:** Process message data input with validation and FIFO management - **Behavior:** - Validate `sha_en = 1` in CFG register - Validate engine started via `hash_start` or `hash_continue` - Validate `hash_process` or `hash_stop` not yet issued - If validation fails: - Reject write - Set `ERR_CODE` to `SwPushMsgWhenDisallowed (0x5)` - Set `INTR_STATE.hmac_err` and assert `intr_hmac_err` - If validation succeeds: - Apply `endian_swap` from CFG if enabled - Push message data to internal FIFO - Handle byte/halfword/word writes via packer logic - Increment message length counter by number of bytes written - Check if complete block accumulated: - SHA-2 256: 512 bits (64 bytes) - SHA-2 384/512: 1024 bits (128 bytes) - If complete block available: - Invoke Botan library hash update with block data - Annotate timing delay (80 or 96 cycles depending on mode) - Check FIFO full condition: - Compare current depth against `MSG_FIFO_DEPTH_SHA256` or `MSG_FIFO_DEPTH_SHA384_512` - If full: Apply TLM back-pressure (block transaction until space available) - Update `STATUS.fifo_full` and `STATUS.fifo_depth`

7.4.10 Status Callback **handle_read_STATUS:** - **Type:** Read callback - **Purpose:** Return live hardware status - **Behavior:** - Compute and return real-time status: - `hmac_idle` (bit 0): Derived from internal state machine (1 = IDLE, 0 = PROCESSING/FINALIZED) - `fifo_empty` (bit 1): Derived from FIFO occupancy (1 = empty, 0 = not empty) - `fifo_full` (bit 2): Derived from FIFO occupancy against depth limit - `fifo_depth` (bits 9:4): Current FIFO entry count (0 to 16/32 depending on mode) - Status dynamically computed, not stored - Reflects current hardware state at time of read

7.5 Registers NOT Requiring Callbacks

ERR_CODE (0x1C): - Pure read-only status register - Value set by other register callbacks - No side-effects on read - No write capability

7.6 Implementation Notes

7.6.1 Port Interface Mapping Callbacks interact with the following output ports:

- **intr_hmac_done** (`sc_out`) - Asserted when hash/HMAC completes
- **intr_fifo_empty** (`sc_out`) - Asserted when FIFO becomes empty (conditional)
- **intr_hmac_err** (`sc_out`) - Asserted on configuration/operational errors
- **alert_fatal_fault** (`sc_out`) - Asserted on TL-UL bus integrity violations

7.6.2 State Machine Dependencies Many callbacks depend on internal state machine states:

- **IDLE:** Engine idle, configuration/key/digest/message length registers writable
- **PROCESSING:** Engine active, configuration/key/digest/message length write-protected, MSG_FIFO writable
- **FINALIZED:** After hash_process/hash_stop issued, MSG_FIFO write-protected

7.6.3 Botan Library Integration The SystemC model delegates cryptographic operations to Botan:

- **Key Setup:** KEY register callbacks provide input to Botan HMAC context initialization
- **Message Update:** MSG_FIFO callback triggers Botan hash update calls at block boundaries
- **Hash Finalization:** CMD.hash_process callback triggers Botan final() and digest retrieval
- **Context Switching:** DIGEST and MSG_LENGTH callbacks save/restore Botan internal state

7.6.4 Error Reporting Callbacks enforce access control and report errors via:

- **ERR_CODE register:** Stores specific error codes (0x1-0x6)
- **INTR_STATE.hmac_err:** Error interrupt flag
- **intr_hmac_err port:** Error interrupt output (when masked by INTR_ENABLE)

Error codes:

- 0x1: SwHashStartWhenShaDisabled
- 0x3: SwUpdateSecretKeyInProcess
- 0x4: SwHashStartWhenActive
- 0x5: SwPushMsgWhenDisallowed
- 0x6: SwInvalidConfig

7.6.5 Performance Considerations Callback implementations model timing annotations:

- **SHA-2 256 block processing:** 80 cycles (configurable via SHA256_BLOCK_CYCLES)
- **SHA-2 384/512 block processing:** 96 cycles (configurable via SHA384_512_BLOCK_CYCLES)
- **HMAC overhead:** 240 additional cycles (configurable via HMAC_EXTRA_CYCLES)

Timing delays annotated using SystemC wait statements based on clock frequency (clk_i port).

8. Modeling Assumptions

8.1 Overview

This section documents 27 key modeling assumptions that define the abstraction level, functional boundaries, and architectural decisions guiding the HMAC SystemC TLM implementation. These assumptions establish the separation between SystemC orchestration (register interface, state management, timing annotation) and Botan library delegation (cryptographic computation).

8.2 Core Architectural Assumptions

Assumption 1: Cryptographic delegation to Botan library

All SHA-2 and HMAC cryptographic computations (compression rounds, message scheduling, padding logic) are delegated to the Botan library. The SystemC model orchestrates data flow, register access, and status management only.

Assumption 2: Transaction-level timing abstraction

Block processing delays are modeled as annotated wait statements (80 cycles for SHA-2 256, 96 cycles for SHA-2 384/512, 240 additional cycles for HMAC mode). No cycle-accurate compression round modeling is implemented.

Assumption 3: Functional state machine model

Engine state is tracked through discrete states (IDLE, PROCESSING, FINALIZED) for access control validation. State transitions are triggered by CMD register writes and hash completion events.

8.3 Message Processing Assumptions

Assumption 4: FIFO depth switching by digest mode

Message FIFO depth is dynamically selected: 16 entries for SHA-2 256, 32 entries for SHA-2 384/512. Depth selection is based on CFG.digest_size runtime configuration.

Assumption 5: Interconnect back-pressure on FIFO full

TLM bus transactions are blocked when MSG_FIFO is full. Blocking is implemented via TLM wait states until FIFO space becomes available.

Assumption 6: Block-boundary processing

Message data is accumulated in FIFO until a complete block is available (512 bits for SHA-2 256, 1024 bits for SHA-2 384/512). The Botan library is invoked only at block boundaries.

Assumption 7: Byte-order conversion abstraction

Endian swap (`endian_swap`, `digest_swap`, `key_swap`) is modeled as data re-ordering at register interface boundaries. No internal bit-level swapping logic is implemented.

8.4 Register Access and Control Assumptions

Assumption 8: Write-only key register protection

`KEY_0` to `KEY_31` registers store values internally but return error on read attempts. Write protection is enforced during `PROCESSING` state with `ERR_CODE` reporting.

Assumption 9: Context switch register writability

`DIGEST_0` to `DIGEST_15` and `MSG_LENGTH` registers are writable only when `STATUS.hmac_idle = 1`. Write attempts during `PROCESSING` state are rejected.

Assumption 10: CMD register self-clearing behavior

Command bits (`hash_start`, `hash_process`, `hash_stop`, `hash_continue`) are automatically cleared after triggering the corresponding action. This is modeled as single-cycle write effect.

Assumption 11: CFG register idle-only updates

Configuration changes (`digest_size`, `key_length`, `hmac_en`, `sha_en`) are validated and applied only in `IDLE` state. Writes are discarded during active hashing with no error reporting (per specification).

8.5 Error Detection Assumptions

Assumption 12: Software error detection only

Six error types are validated in software access patterns (`SwHashStartWhenShaDisabled`, `SwHashStartWhenActive`, `SwUpdateSecretKeyInProgress`, `SwPushMsgWhenDisallowed`, `SwInvalidConfig`). No bus integrity error modeling is implemented beyond alert generation.

Assumption 13: Configuration validation on `hash_start`

Digest size and key length combinations are validated when the `hash_start` command is issued. Invalid configurations (`SHA2_None`, `Key_None` for HMAC, `Key_1024` for SHA-2 256) are rejected with `ERR_CODE.SwInvalidConfig`.

8.6 Interrupt and Status Assumptions

Assumption 14: Conditional `fifo_empty` interrupt

`INTR_STATE.fifo_empty` is asserted only when FIFO transitions from full to empty. It is not asserted on every empty condition to avoid interrupt flooding.

Assumption 15: Immediate interrupt output update

Interrupt ports (`intr_hmac_done`, `intr_fifo_empty`, `intr_hmac_err`) are updated immediately when `INTR_STATE` or `INTR_ENABLE` is modified. This is modeled as combinational output = `INTR_STATE & INTR_ENABLE`.

Assumption 16: Live STATUS register reflection

The STATUS register (`hmac_idle`, `fifo_full`, `fifo_empty`, `fifo_depth`) is dynamically computed from internal state on every read. No static storage is maintained for status bits.

8.7 Context Switching Assumptions**Assumption 17: Block-boundary context switch constraint**

Context save/restore operations (`hash_stop`/`hash_continue`) are restricted to complete message block boundaries. Partial block switching is not supported.

Assumption 18: Software-managed CFG and KEY preservation

CFG register value and KEY register values cannot be read back. Software is responsible for maintaining separate copies for context restoration.

Assumption 19: DIGEST and MSG_LENGTH context state

`DIGEST_0` to `DIGEST_15` and `MSG_LENGTH` registers capture complete hash state for context switching. Botan library state is serialized/deserialized through these registers.

8.8 Security Feature Assumptions**Assumption 20: WIPE_SECRET overwrites internal state**

`WIPE_SECRET` register write clears KEY storage, intermediate hash values, DIGEST registers, and message scheduling array. This is modeled as immediate state reset with the provided wipe value.

Assumption 21: TL-UL bus integrity alert abstraction

Fatal fault alert (`alert_fatal_fault`) is modeled as software-testable output via `ALERT_TEST` register. No actual bus integrity checking is implemented in the TLM model.

8.9 Performance and Timing Assumptions**Assumption 22: Observable block latency only**

Timing annotations are limited to software-observable block processing latencies. Sub-block internal compression rounds are not modeled.

Assumption 23: HMAC two-round overhead

HMAC mode adds 240 fixed cycles (80 inner key, 80 outer key, 80 first-round result processing) to total hash completion time. This is modeled as sequential Botan calls with annotated delays.

8.10 Data Transfer Assumptions

Assumption 24: MSG_FIFO window address abstraction

All writes to MSG_FIFO address window (0x1000-0x1FFF) are directed to a single FIFO entry point. The specific address within the window is ignored except for the lower 2 bits for sub-word write alignment.

Assumption 25: Sub-word write packing

Byte and halfword writes to MSG_FIFO are coalesced into 32-bit words via packer logic. Message length is tracked at byte granularity regardless of write size.

8.11 Excluded Features

Assumption 26: No future pipelined optimization

The current model does not implement pipelined message feeding during compression rounds (mentioned as future enhancement in documentation). Messages are processed strictly sequentially.

Assumption 27: No low-power or clock gating

Power management features (clock gating, power domains) are not modeled. Only functional behavior at TLM abstraction level is implemented.

8.12 Summary

These 27 assumptions define the functional boundary between:

- **SystemC Orchestration:** Register interface, state management, timing annotation, error detection, interrupt generation, FIFO management
- **Botan Library Delegation:** Cryptographic computation, compression functions, message scheduling, padding, key derivation, hash finalization

All assumptions are derived from HMAC feature classification, configuration parameters, register specifications, callback requirements, and theory of operation documentation.

9. Summary and Conclusions

9.1 Design Summary

This document provides a comprehensive high-level design specification for the HMAC SystemC TLM2.0 peripheral. The design encompasses:

Functional Scope: - Dual-mode operation: SHA-2 hashing (256/384/512-bit) and HMAC message authentication - Flexible message processing with 4KB FIFO window and DMA-friendly interface - Configurable key lengths (128 to 1024 bits) with security protection - Context switching support for time-interleaved multi-stream processing - Comprehensive error detection and interrupt generation - Performance-accurate timing annotations for software development

Architectural Characteristics: - Transaction-level abstraction suitable for virtual platform integration - Clean separation between orchestration (SystemC) and computation (Botan library) - Software-visible functional behavior fully modeled - Hardware resource management (FIFO depth, register arrays, timing parameters) - State machine-based access control and validation

Integration Interfaces: - TL-UL register bus for memory-mapped access (59 registers, 8KB address space) - Three interrupt outputs for completion and error notification - Fatal alert output for security violations - Clock and reset inputs for timing and initialization

9.2 Key Design Decisions

1. Botan Library Integration

The most critical architectural decision is delegating all cryptographic computations to the proven opensource Botan library. This provides:

- Correctness through established cryptographic implementations
- Maintainability by separating concerns (orchestration vs. computation)
- Appropriate abstraction for virtual platform modeling
- Focus on software-visible behavior rather than internal algorithms

2. Transaction-Level Timing

Block processing latencies are modeled with annotated timing delays rather than cycle-accurate execution:

- SHA-2 256: 80 cycles per 512-bit block
- SHA-2 384/512: 96 cycles per 1024-bit block
- HMAC: Additional 240 cycles for two-round processing

This provides sufficient accuracy for software performance estimation while maintaining appropriate TLM abstraction level.

3. State Machine Access Control

A three-state model (IDLE, PROCESSING, FINALIZED) enforces register access protection:

- Configuration and key registers writable only when IDLE
- Context switch registers (DIGEST, MSG_LENGTH) writable only when IDLE
- Message FIFO writable only during PROCESSING

This ensures software follows correct command sequences and prevents race conditions.

4. Comprehensive Error Detection

Six distinct error types are detected and reported:

- Command sequence violations (hash start when disabled/active)
- Configuration errors (invalid digest size, key length combinations)
- Key update violations (writes during active hashing)
- Message FIFO write violations (writes when operation not allowed)

Each error generates detailed ERR_CODE value and triggers error interrupt.

9.3 Implementation Guidance

For Model Developers:

1. Implement all 60 register callbacks according to specifications in Section 7
2. Use Botan library API for all cryptographic operations (SHA-2, HMAC)
3. Maintain three-state machine for access control validation
4. Implement FIFO management with dynamic depth selection (16/32 entries)
5. Apply TLM timing annotations based on configuration parameters
6. Generate interrupts and alerts according to callback specifications

For Test Developers:

1. Verify all operational modes (SHA-2 256/384/512, HMAC with various key lengths)
2. Test context switching at block boundaries with multiple message streams
3. Validate all error detection paths with negative test cases
4. Verify timing characteristics match annotated cycle counts
5. Test FIFO back-pressure and DMA-style burst writes
6. Verify interrupt and alert generation under all conditions

For Integration Engineers:

1. Connect tl_socket to platform memory bus fabric
2. Route interrupt outputs to interrupt controller
3. Connect alert output to security alert aggregator
4. Provide clock frequency value to clk_i port for accurate timing
5. Connect reset signal to platform reset controller
6. Allocate 8KB address space starting at configured base address

9.4 Validation Strategy

The design supports comprehensive validation through:

Functional Coverage: - All register access types (RO, WO, RW, W1C, r0w1c)
- All digest modes (SHA-2 256/384/512) with and without HMAC - All key

lengths (128/256/384/512/1024 bits) - All command sequences (start, process, stop, continue) - All error conditions with ERR_CODE verification

Performance Verification: - Timing measurements against annotated cycle counts - Throughput validation for different digest modes - HMAC overhead verification (240 cycles)

Security Validation: - Write-only key register protection - Secret wipe functionality - Alert generation on bus integrity violations

Interoperability Testing: - Comparison against reference Botan library outputs - Cross-validation with hardware RTL simulation results - Compliance with HMAC RFC 2104 and SHA-2 FIPS 180-4 standards

9.5 Future Enhancements

Potential future enhancements identified but not currently implemented:

1. **Pipelined Message Feeding:** Allow message FIFO writes during compression rounds for improved throughput (mentioned in documentation as future optimization)
2. **Additional Hash Algorithms:** Support for SHA-3 or other hash functions through modular Botan integration
3. **DMA Engine Integration:** Hardware DMA controller for autonomous message transfer without software intervention
4. **Power Management:** Clock gating and power domain modeling for low-power analysis (beyond current TLM scope)
5. **Cryptographic Acceleration:** Hardware acceleration of specific cryptographic primitives while maintaining Botan delegation architecture

9.6 Conformance and Standards

The HMAC peripheral design conforms to:

- **TLM2.0:** SystemC Transaction Level Modeling standard (IEEE 1666)
- **HMAC:** RFC 2104 (Keyed-Hashing for Message Authentication)
- **SHA-2:** FIPS 180-4 (Secure Hash Standard)
- **Botan:** Opensource cryptographic library API and conventions

9.7 Conclusion

This high-level design specification provides a complete and consistent definition of the HMAC SystemC TLM peripheral. The design achieves:

- **Functional Completeness:** All software-visible features are specified and will be modeled
- **Architectural Clarity:** Clean separation between orchestration and cryptographic computation

- **Implementation Guidance:** Detailed specifications enable straightforward model development
- **Verification Support:** Comprehensive coverage points identified for validation
- **Integration Readiness:** Standard interfaces enable seamless virtual platform integration

The design is ready for detailed implementation, with all major architectural decisions documented and all functional requirements specified. The use of the established Botan library for cryptographic operations ensures correctness while allowing the SystemC model to focus on transaction-level orchestration behavior.

Document Control

Version	Date	Author	Description
1.0	2025-10-23	SystemC TLM Design Team	Initial high-level design specification

End of Document